

WIA2005: Algorithm Design and Analysis

Lecture 4: String Matching (Asynchronous)

Asmiza A. Sani

Semester 1, Session 2024/25

Learning objectives

- Know string matching algorithm
 - Understand the string-matching problem and its relevance.
 - Explain and implement Naïve, Finite-automaton, Knuth-Morris-Pratt (KMP), and Rabin-Karp algorithms.
 - Analyse the time and space complexity of each algorithm.
 - Apply appropriate algorithms to real-world problems.
 - Extra: Tries

Introduction

- Text-editing programs frequently need to find all **pattern occurrences** in the text.
- Typically, the text is a document being edited, and the pattern searched for is a particular **word supplied by the user**.
- Efficient algorithms for this problem—“string matching”—can greatly improve the **responsiveness** of the text-editing program.

Among their many other applications string-matching algorithms

Search for particular patterns in DNA sequences.

Internet search engines also use them to find web pages that are relevant to queries.

String matching problem

- String-matching problems are formalized as follows:
 - Assume that the text is an array $T[1..n]$ of length n and that the pattern is an array $P[1..m]$ of length $m \leq n$.
 - Further assume that the elements of P and T are characters drawn from a finite alphabet Σ .
 - For example, we may have $\Sigma = \{0,1\}$ or $\Sigma = \{a, b, c, \dots, z\}$.
 - The character arrays P and T are often called **strings** of characters.

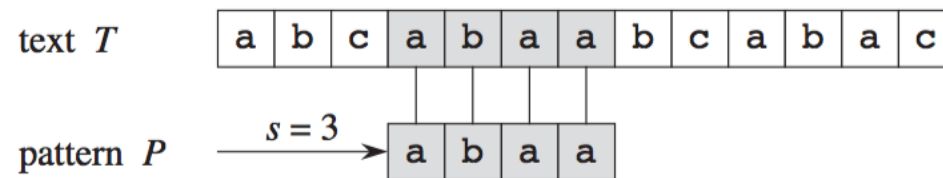
Asynchronous Lecture Instruction

- Whenever there is a link on the top of the slides (Highlighted yellow), please watch the video provided first.
- Then go through the rest of the slides.

<https://youtu.be/yKhPWrdA6U8?si=nxXqZnSeUqmPKBoP>

Naïve (Brute-force) string-matching algorithm

- An example of the string-matching problem, where we want to find all occurrences of the pattern $P = \text{abaa}$ in the text $T = \text{abcabaabcabac}$.
- The pattern occurs only once in the text, at shift $s = 3$, which we call a valid shift.
- A vertical line connects each character of the pattern to its matching character in the text, and all matched characters are shaded.

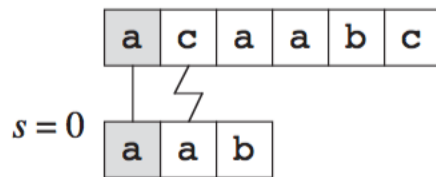


Naive string-matching pseudocode

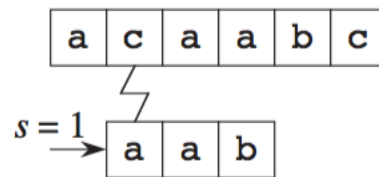
- The naive algorithm finds all valid shifts using a loop that checks the condition $P = [1..m] = T[s + 1..s + m]$ for each of the $n - m + 1$ possible values of s . No preprocessing phase.
- Constant extra space needed;
- Always shifts the window by exactly 1 position to the right;
- Comparisons can be done in any order;
- Searching phase in $O(mn)$ time complexity;
- $2n$ expected text characters comparisons.

NAIVE-STRING-MATCHER(T, P)

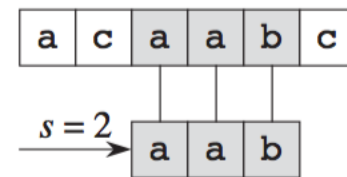
```
1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print "Pattern occurs with shift"  $s$ 
```



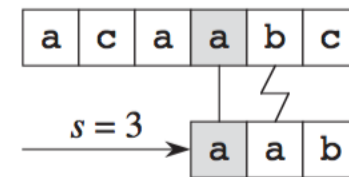
(a)



(b)



(c)



(d)

<https://youtu.be/5KJtYMjeY4U?si=QzQhY8Vs75aP2mKG>

String matching with finite automata

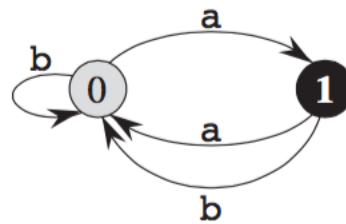
- Many string-matching algorithms build a finite automaton—a simple machine for processing information—that scans the text string T for all occurrences of the pattern P .
- These string-matching automata are very efficient: they examine each text character *exactly once*, taking constant time per text character.
 - especially for regular expressions used in compiler (parser)

Finite automata

- A *finite automaton (plural of automata)* M , is a 5-tuple $(Q, q_0, A, \Sigma, \delta)$ where:
 - Q is a finite set of *states*,
 - $q_0 \in Q$ is the *start state*,
 - $A \subseteq Q$ is a distinguished set of *accepting states*,
 - Σ is a finite *input alphabet*,
 - δ is a function from $Q \times \Sigma$ into Q , called the *transition function* of M .

state	input	
	a	b
0	1	0
1	0	0

(a)



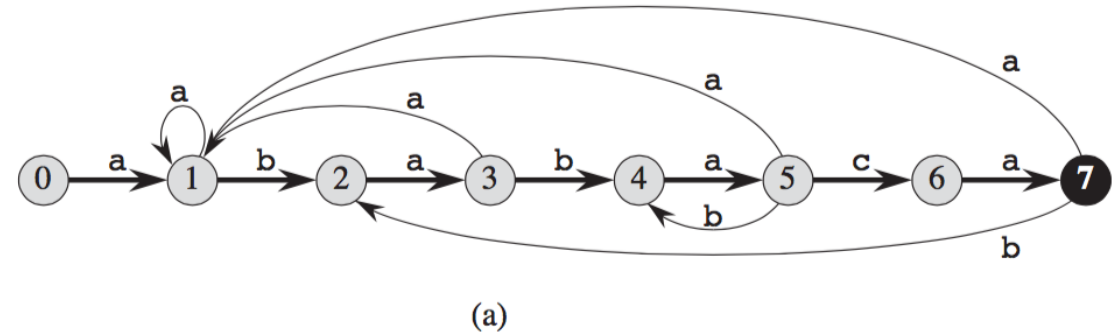
(b)

For example, on input $abaaa$, including the start state, this automaton enters the sequence of states $\langle 0, 1, 0, 1, 0, 1 \rangle$, and so it accepts this input.

For input $abbaa$, it enters the sequence of states $\langle 0, 1, 0, 0, 1, 0 \rangle$, and so it rejects this input.

String matching automata

- For a given pattern P , we construct a string-matching automaton in a preprocessing step before using it to search the text string.
- In order to specify the string-matching automaton corresponding to a given pattern $P[1..m]$ we first define an auxiliary function σ , called the **suffix function** corresponding to P .



state	input			P
	a	b	c	
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

(b)

i	—	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	—	a	b	a	b	a	b	a	c	a	b	a
state $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

(c)

Finite-automaton string-matching pseudocode

COMPUTE-TRANSITION-FUNCTION(P, Σ)

```
1   $m = P.length$ 
2  for  $q = 0$  to  $m$ 
3      for each character  $a \in \Sigma$ 
4           $k = \min(m + 1, q + 2)$ 
5          repeat
6               $k = k - 1$ 
7          until  $P_k \sqsupseteq P_q a$ 
8           $\delta(q, a) = k$ 
9  return  $\delta$ 
```

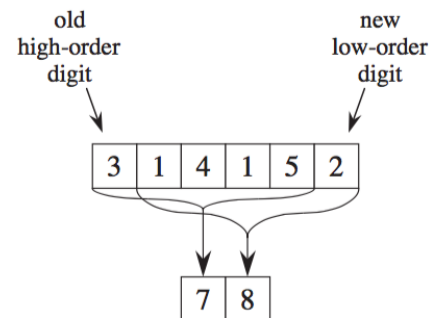
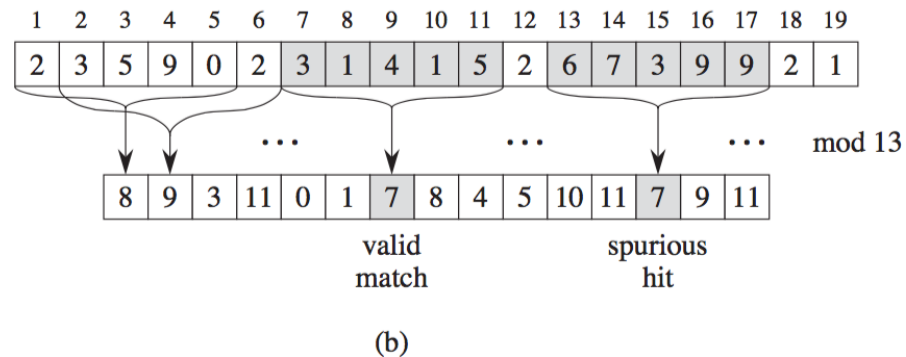
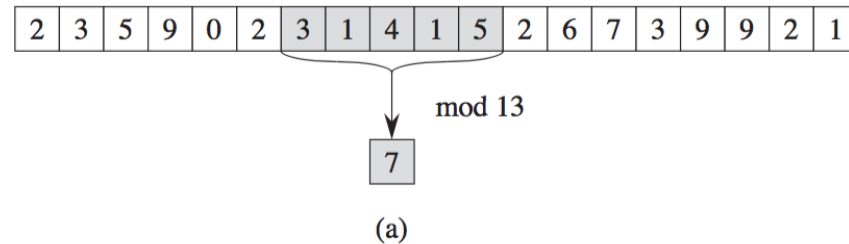
FINITE-AUTOMATON-MATCHER(T, δ, m)

```
1   $n = T.length$ 
2   $q = 0$ 
3  for  $i = 1$  to  $n$ 
4       $q = \delta(q, T[i])$ 
5      if  $q == m$ 
6          print "Pattern occurs with shift"  $i - m$ 
```

https://youtu.be/yFHV7weZ_as?si=2bzdiV_pwxkLNTVm

Rabin-Karp algorithm

- Rabin and Karp proposed a string-matching algorithm that performs well in practice and that also generalizes to other algorithms for related problems, such as two-dimensional pattern matching.
- Using hash (Rolling hash)
- The Rabin-Karp algorithm uses $O(m)$ preprocessing time, and its worst-case running time is $O((n-m+1)m)$



$$\begin{aligned} 14152 &\equiv (31415 - 3 \cdot 10000) \cdot 10 + 2 \pmod{13} \\ &\equiv (7 - 3 \cdot 3) \cdot 10 + 2 \pmod{13} \\ &\equiv 8 \pmod{13} \end{aligned}$$

Rabin-Karp Pseudocode

RABIN-KARP-MATCHER(T, P, d, q)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $h = d^{m-1} \bmod q$ 
4   $p = 0$ 
5   $t_0 = 0$ 
6  for  $i = 1$  to  $m$            // preprocessing
7       $p = (dp + P[i]) \bmod q$ 
8       $t_0 = (dt_0 + T[i]) \bmod q$ 
9  for  $s = 0$  to  $n - m$        // matching
10     if  $p == t_s$ 
11         if  $P[1..m] == T[s + 1..s + m]$ 
12             print "Pattern occurs with shift"  $s$ 
13     if  $s < n - m$ 
14          $t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q$ 
```

https://youtu.be/q4_90fOoS-s?si=4QegDtr07hFVcvoV

Knuth-Morris-Pratt (KMP) algorithm

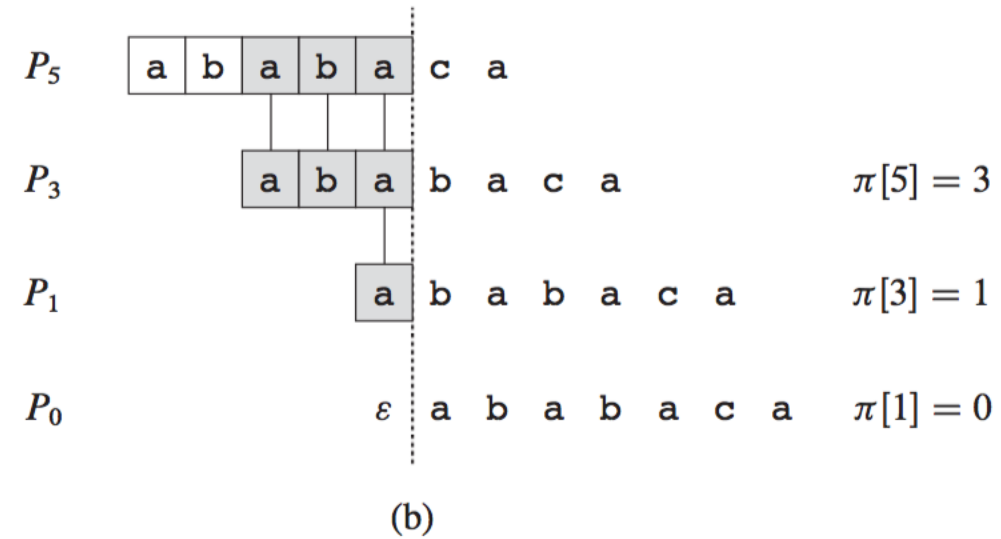
- Unlike the naïve approach (which can re-check characters unnecessarily), KMP uses information about the pattern to skip unnecessary comparisons, making it **faster in the worst case**.
- When a mismatch occurs, instead of shifting the pattern by one character (like the naïve method), KMP uses a precomputed table (called the LPS array) to know how far to shift the pattern, based on what is already known.
- Preprocessing time takes $O(m)$, where m is the length of the pattern.
- Searching time takes $O(n)$, where n is the length of the text.
- Overall running time complexity is $O(n + m)$

Prefix table (LPS Array)

- Complete prefix function for the pattern ababaca:

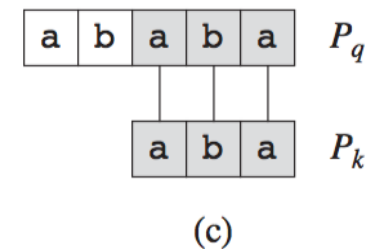
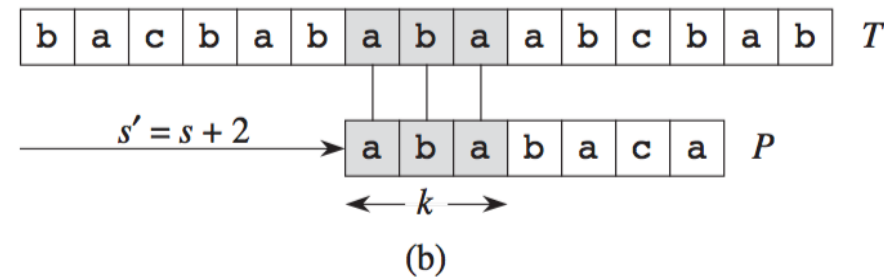
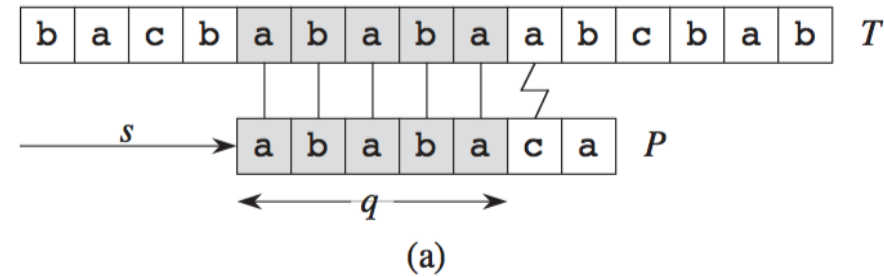
i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

(a)



Prefix function for a pattern

- The prefix function π for a pattern encapsulates knowledge about how the pattern matches against shifts of itself.
- We can take advantage of this information to avoid testing useless shifts in the naive pattern-matching algorithm and to avoid pre-computing the full transition function δ for a string-matching automaton.



KMP pseudocode

- The pseudocode below gives the Knuth-Morris-Pratt matching algorithm as the procedure KMP-MATCHER.
- For the most part, the procedure follows from FINITE-AUTOMATON-MATCHER.
- KMP-MATCHER calls the auxiliary procedure COMPUTE-PREFIX-FUNCTION to compute

KMP-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```

COMPUTE-PREFIX-FUNCTION(P)

```
1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11  return  $\pi$ 
```

Pre-processing and matching time

Algorithm	Preprocessing time	Matching time
Naive	0	$O((n - m + 1)m)$
Rabin-Karp	$\Theta(m)$	$O((n - m + 1)m)$
Finite automaton	$O(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$

Pre-processing text approach

- So far, the previous algorithm are pre-processing patterns.
- Another fast approach to finding text is to pre-process text.
 - Text does not change and many search queries.
 - Dictionary
- String matching using Trie.

Extra: Tries

https://youtu.be/zljfhVPRZCg?si=uWf_NRuu9Px2G1AR

Tries

- For representing a collection of Strings, such as all the words in a text.
- A tries supports pattern matching queries in time proportional to the pattern size.
- Properties of a tries:
 - A multi-way tree
 - Each node has from 1 to n children.
 - Each edge of the tree is labelled with a character.
 - Each leaf node corresponds to the stored string, which is a concatenation of characters on a path from the root to this node.

Types of Tries

Standard Tries

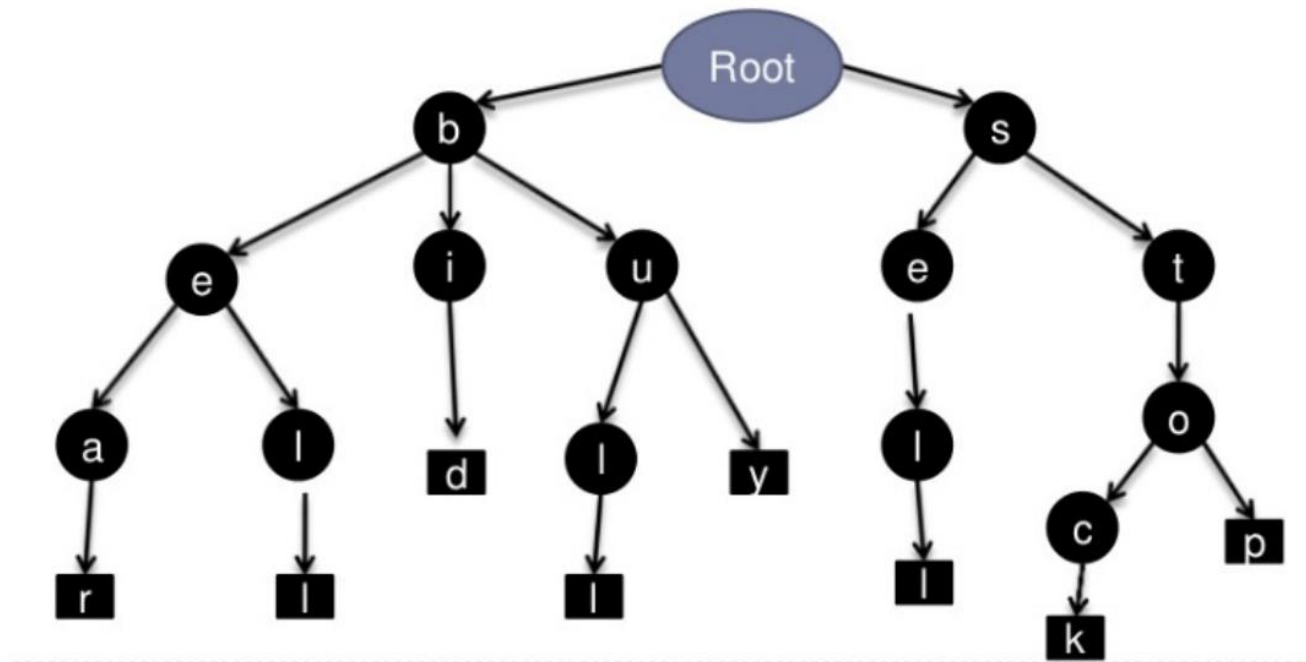
Compressed/Compact
Tries

Suffix Tries

Standard Tries

- The standard trie for a set of strings S is an ordered tree such that:
 - Each node but the root is labelled with a character.
 - The children of a node are alphabetically ordered.
 - The paths from the external nodes to the root yield the strings of S .

Example of a Standard Tries



- $S = \{\text{bear, bell, bid, bull, buy, sell, stock, stop}\}$

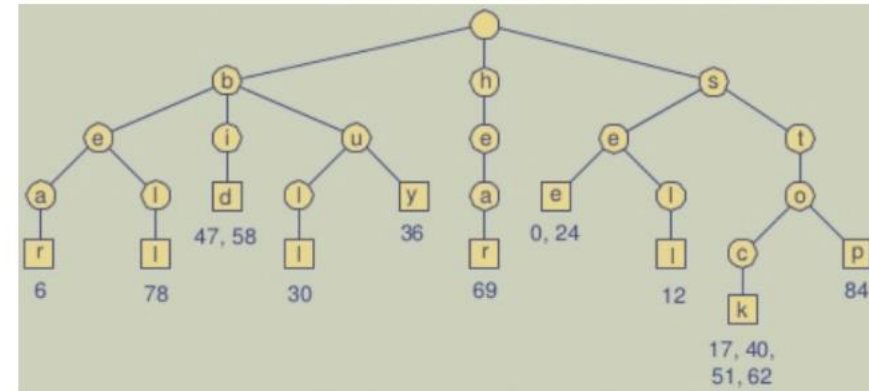
Time/Space complexity of Standard Tries

- A standard trie uses $O(n)$ space and supports searches, insertions and deletions in time $O(dm)$, where:
 - n - total size of the strings in S
 - m - size of the string parameter of the operation
 - d - size of the alphabet

Word matching with a Trie

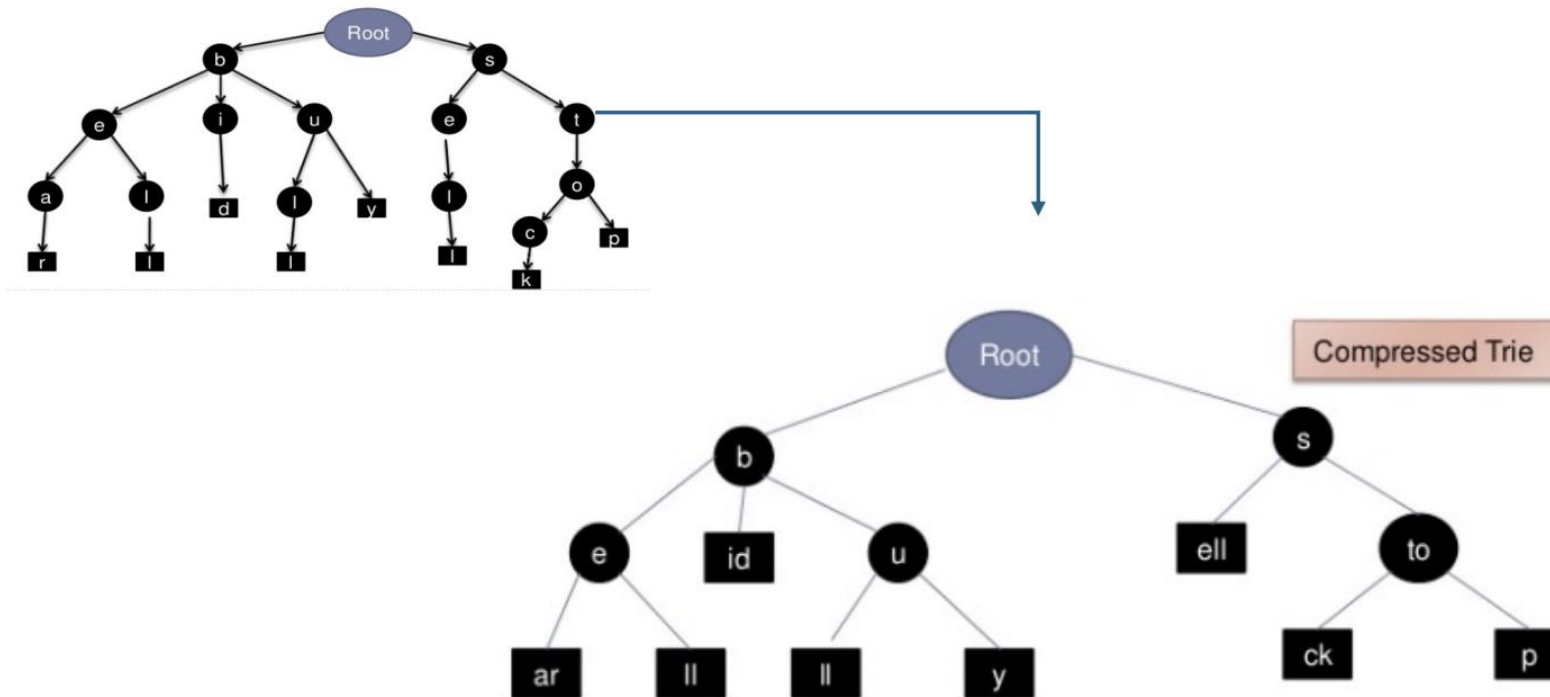
- We insert the words of the text into a trie.
- Each leaf stores the occurrences of the associated word in the text.

s	e	e		a		b	e	a	r	?		s	e	l	l		s	t	o	c	k	!	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
s	e	e		a		b	u	l	l	?		b	u	y		s	t	o	c	k	!		
24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	
b	i	d		s	t	o	c	k	!		b	i	d		s	t	o	c	k	!			
47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68		
h	e	a	r		t	h	e		b	e	l	l	?		s	t	o	p	!				
69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88				



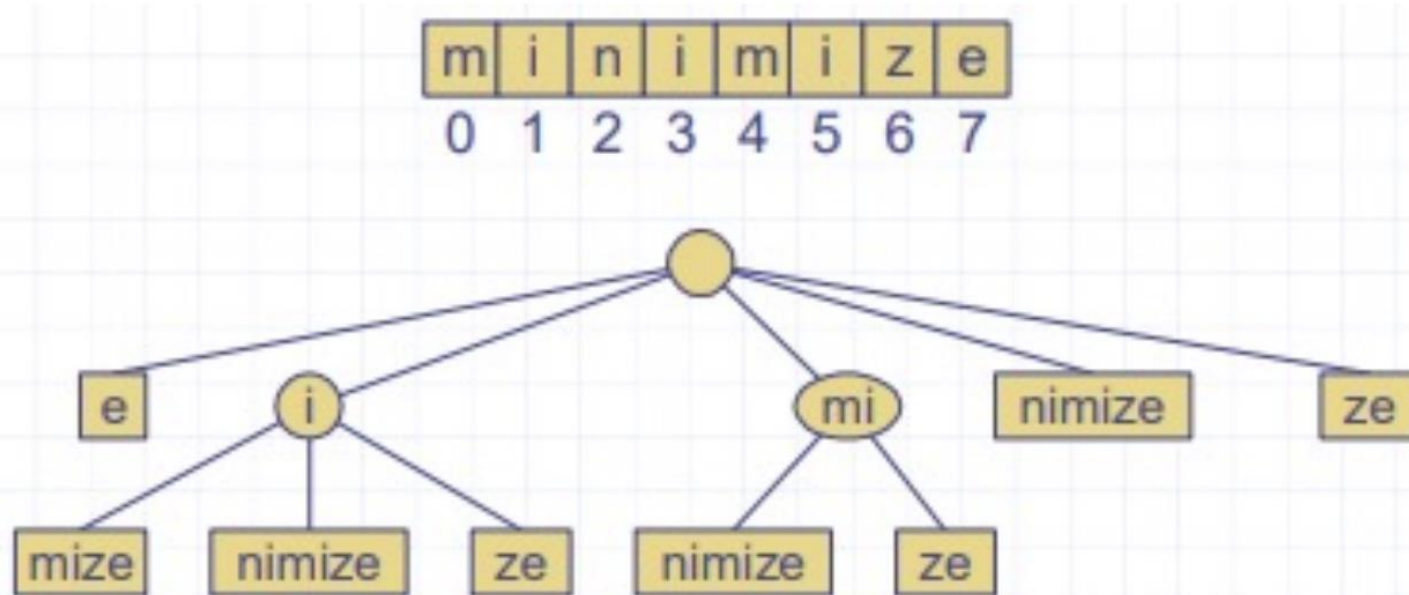
Compressed Trie

- Tries with nodes of degree at least 2.
- Obtained by standard tries by compressing chains of redundant nodes.



Suffix Tries

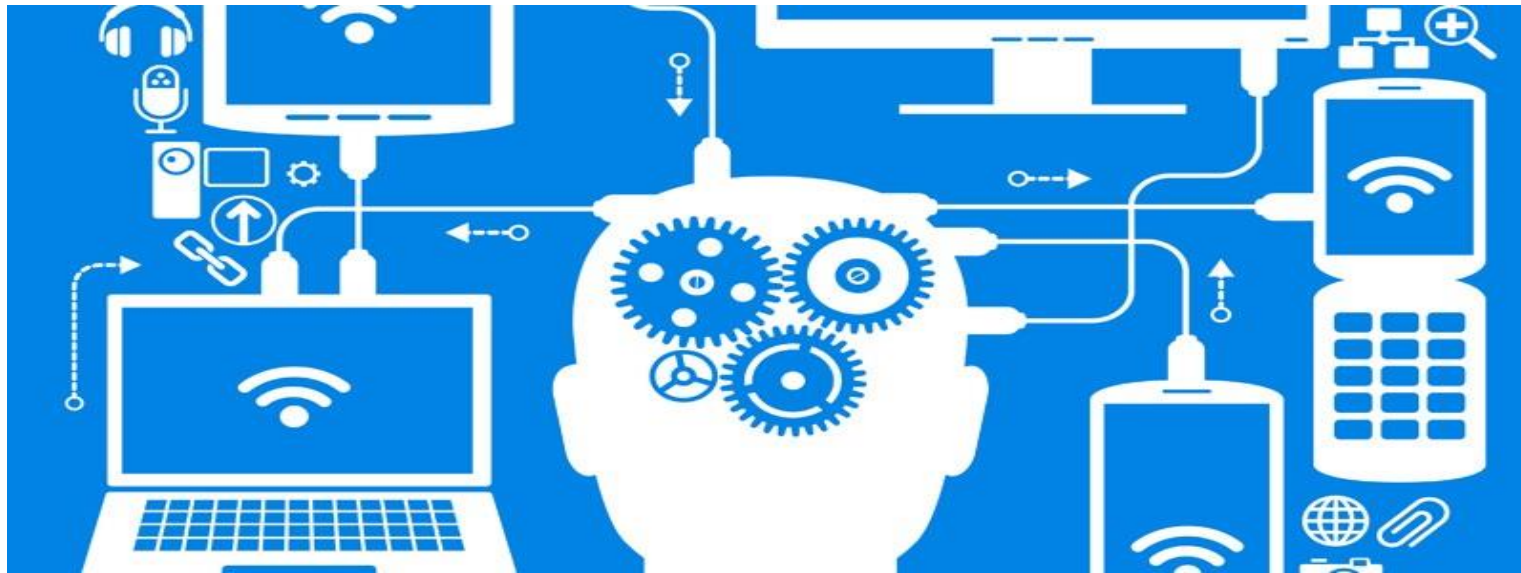
- A suffix trie is a compressed trie for all the suffixes of a text.
- Suffix trie are a space-efficient data structure to store a string that allows many kinds of queries to be answered quickly.



Analysis of Suffix Tries

- Compact representation of the suffix trie for a string S of size n from an alphabet of size d
 - uses $O(n)$ space.
 - supports arbitrary pattern matching queries in S in $O(dm)$ time, where m is the size of the pattern.
 - can be constructed in $O(n)$ time.

In the next lecture..



Lecture 5: Divide and Conquer Algorithm

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein. 2009. Introduction to Algorithms, 3rd edition. MIT Press.